

Fondamenti di Analisi dei Dati

from **data analysis** to **predictive techniques**

Prof. Antonino Furnari (antonino.furnari@unict.it)

Corso di Studi in Informatica

Dip. di Matematica e Informatica

Università di Catania



Università
di Catania

Data Representation

How data turns from a table into geometric points in a feature space, and how representation functions unlock what comes next

From Data Tables to Feature Spaces

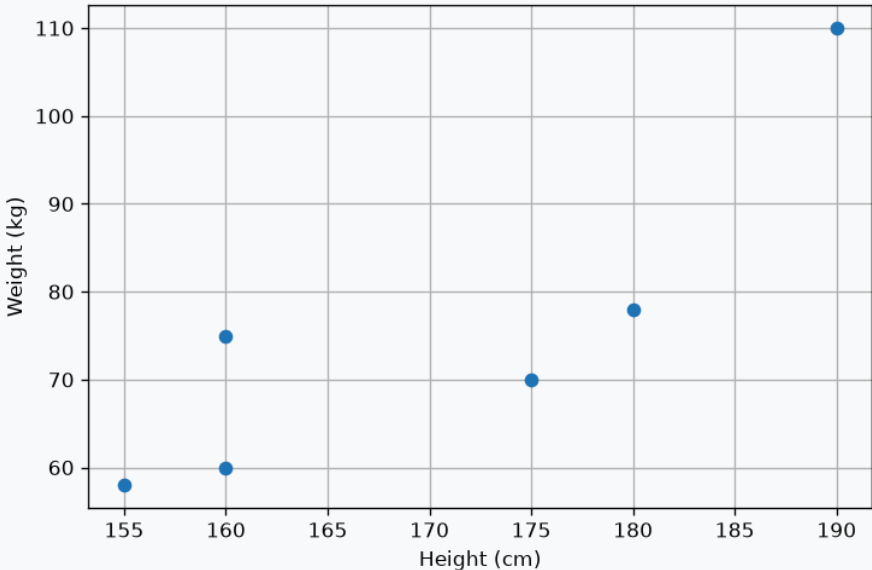
Traditional View

Data as tables with named variables and observations. The focus is on interpreting the values of each variable and what they mean.

Subject	Height (cm)	Weight (kg)
1	175	70
2	160	60
3	180	78
4	160	75
5	155	58
6	190	110

Geometric View

Data as d -dimensional points in a vector space. Variables become **features**, and mathematical operations — distances above all — become available.



Each row of the table is a data point, and the six points can be drawn in the plane.

When we work with many variables, the geometric interpretation becomes essential, and we stop paying attention to the names: a dataset of 100×100 pixel images is a collection of **10 000-dimensional vectors**, where each dimension is one pixel.



Feature Extraction and Representation Functions

Given this interpretation of variables as features, it is common to talk about **feature extraction** for any process transforming the data $\mathbf{x} \in \mathbb{R}^d$ into another form $\mathbf{y} \in \mathbb{R}^m$. In this context, a function

$$f : \mathbb{R}^d \rightarrow \mathbb{R}^m$$

is called a "**feature extraction function**" (or feature extractor) or a "**representation function**". We refer to the process of mapping data with a representation function as **data representation**.

We also refer to $\mathbb{R}^d$ and $\mathbb{R}^m$ as "**feature spaces**" or "**representation spaces**", since they are **vector spaces** to which the data points — also called **feature vectors** — belong.

- Because observations are vectors living in vector spaces, every basic property of vectors from linear algebra applies here. The next two slides recall the two we will use constantly: norms and metrics.



Understanding Norms: Measuring Vector Magnitude

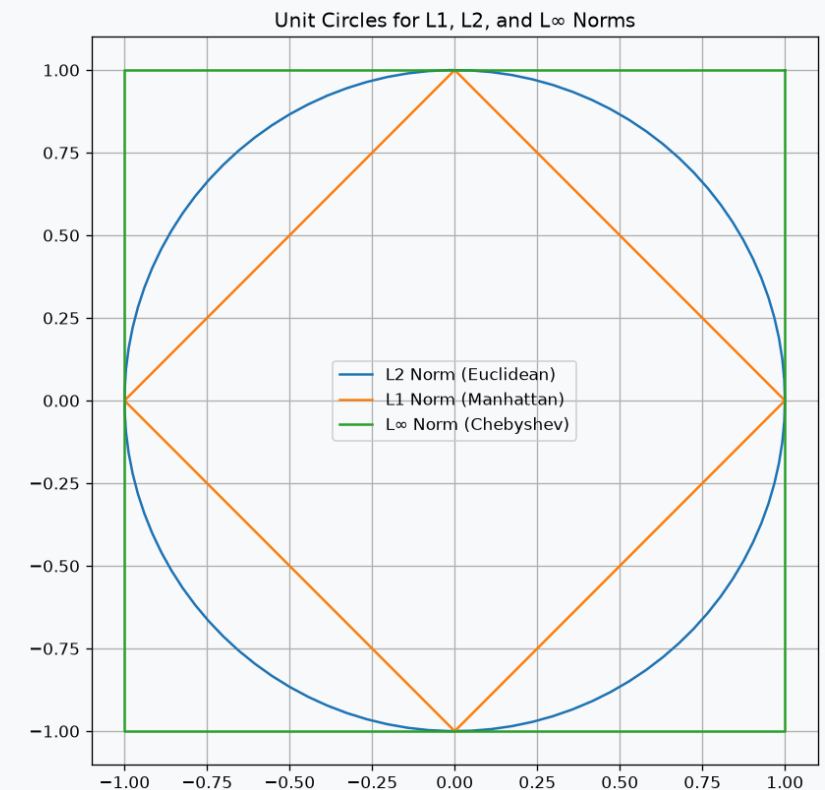
Given a vector space S , a norm p is a function mapping vectors to non-negative real numbers, measuring how far the vector is from the origin:

$$p : S \rightarrow \mathbb{R}_0^+$$

The **L-p family** gives different ways to quantify magnitude, with p the parameter that defines the behaviour of the norm:

$$||\mathbf{x}||_p = \left(\sum_{i=1}^d |x_i|^p \right)^{1/p}$$

For the L2 norm we usually drop the subscript, $||\mathbf{x}||_2 = ||\mathbf{x}||$, and write the squared norm as $||\mathbf{x}||^2 = \sum_i x_i^2$.



Each shape is made of the vectors of unit norm according to one norm.

L2 Norm (Euclidean)

$$||\mathbf{x}||_2 = \sqrt{\sum_{i=1}^d x_i^2}$$

The "magnitude" of a vector: the straight-line distance from the origin to the point.

L1 Norm (Manhattan)

$$||\mathbf{x}||_1 = \sum_{i=1}^d |x_i|$$

The sum of the absolute values of the components. Named after city blocks.

L-infinity Norm (Maximum)

$$||\mathbf{x}||_\infty = \max\{|x_i|\}_{i=1}^d$$

The largest absolute value among the components.



Metrics: Measuring Distance Between Points

A metric is a function

$$m : S \times S \rightarrow \mathbb{R}_0^+$$

that measures the distance between points. A valid metric must satisfy four properties:

1. The distance of a point from itself is zero: $m(\mathbf{x}, \mathbf{x}) = 0$;
2. the distance between two distinct points is always positive: $m(\mathbf{x}, \mathbf{y}) > 0, \forall \mathbf{x}, \mathbf{y} \in S, \mathbf{x} \neq \mathbf{y}$;
3. symmetry: $m(\mathbf{x}, \mathbf{y}) = m(\mathbf{y}, \mathbf{x})$;
4. triangle inequality: $m(\mathbf{x}, \mathbf{y}) \leq m(\mathbf{x}, \mathbf{z}) + m(\mathbf{z}, \mathbf{y}), \forall \mathbf{x}, \mathbf{y}, \mathbf{z} \in S$.

From the L-p norms we derive **a family of metrics**:

$$L_p(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|_p = \left(\sum_{i=1}^d |x_i - y_i|^p \right)^{1/p}$$

01

L2 Distance (Euclidean)

$$L_2(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^d (x_i - y_i)^2}$$

The straight-line distance between two points in space.

02

L1 Distance (Manhattan)

$$L_1(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^d |x_i - y_i|$$

Distance measured along axis-aligned paths, like navigating city blocks.

03

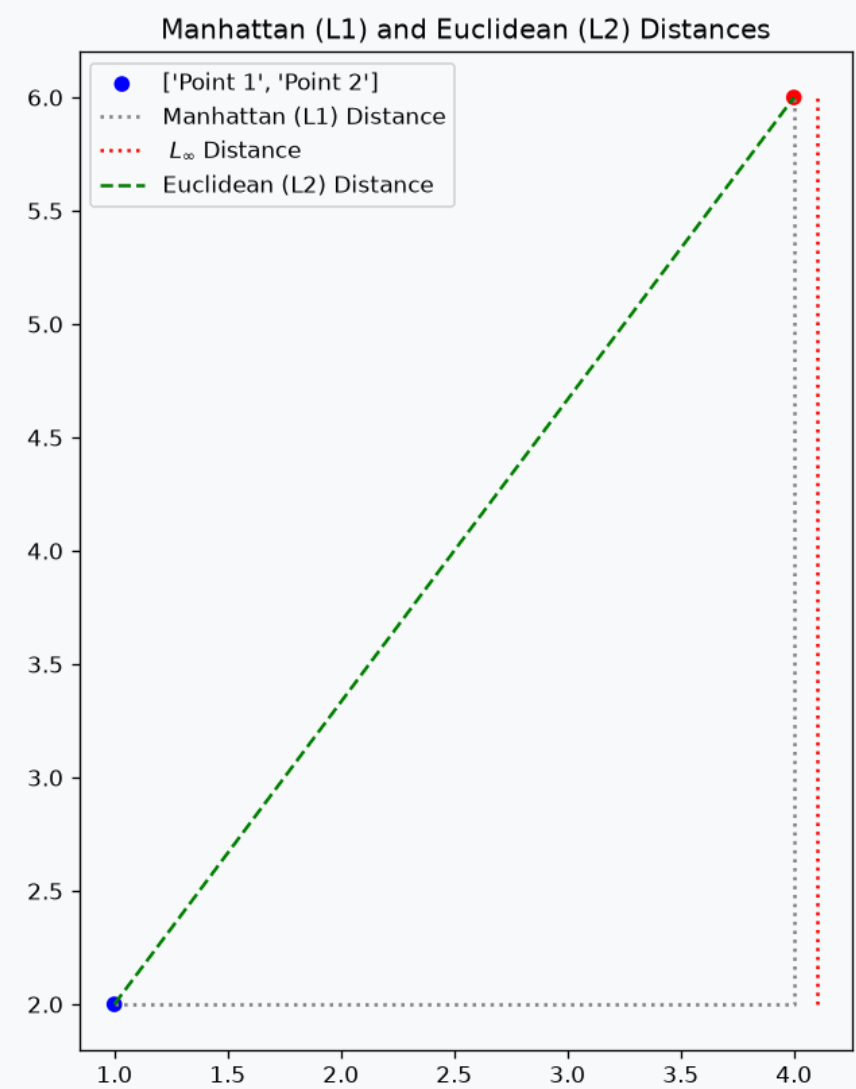
L_∞ Distance

$$L_\infty(\mathbf{x}, \mathbf{y}) = \max_{i=1}^d \{|x_i - y_i|\}$$

The largest difference across any single dimension.



Manhattan vs. Euclidean Distance



The three distances between the same two points, (1, 2) and (4, 6):

7.00

Manhattan (L1)

The path a taxi must drive
along grid streets

5.00

Euclidean (L2)

The straight line, "as the
crow flies"

4.00

L_∞ (Chebyshev)

The largest coordinate
difference

While the Euclidean distance measures the length of the straight segment joining the two points, the Manhattan distance is the dashed path: what a taxi driver has to drive in Manhattan — or any other city built on square blocks — to reach the destination.



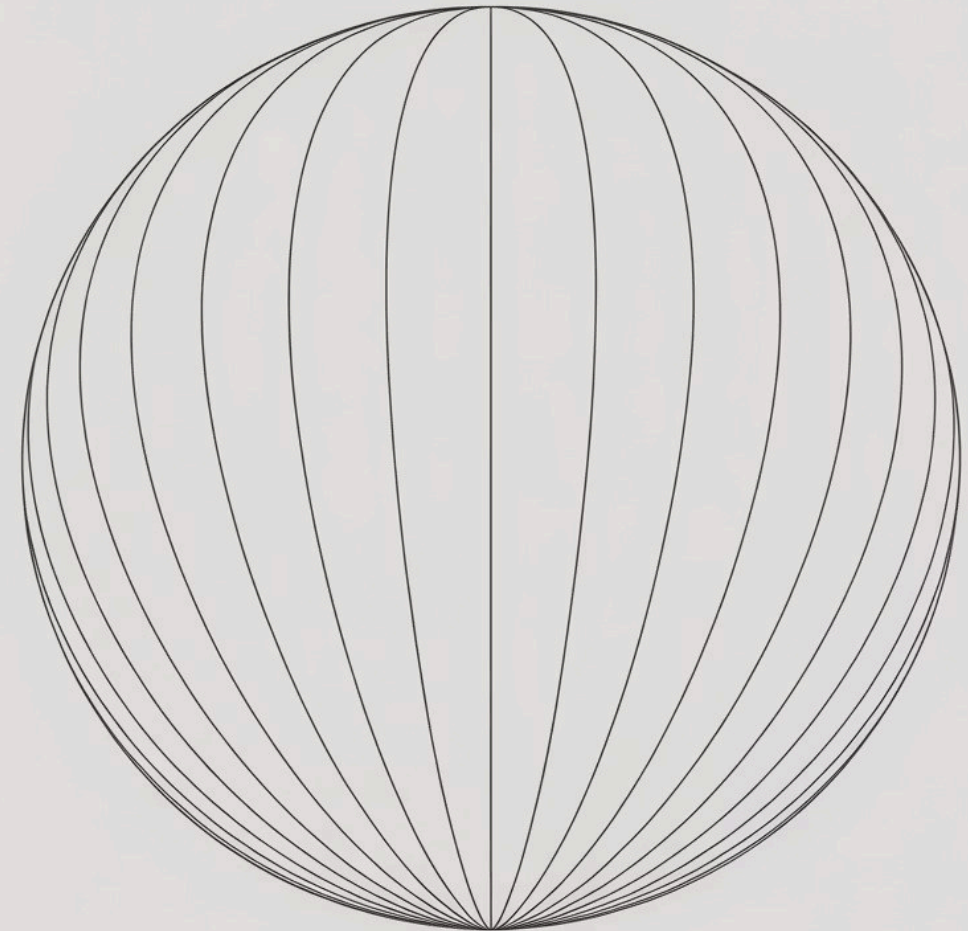
Vector Normalization

Scaling feature vectors to unit length, known as **normalization**, is a common preprocessing step. It makes features contribute on comparable terms regardless of their original scale.

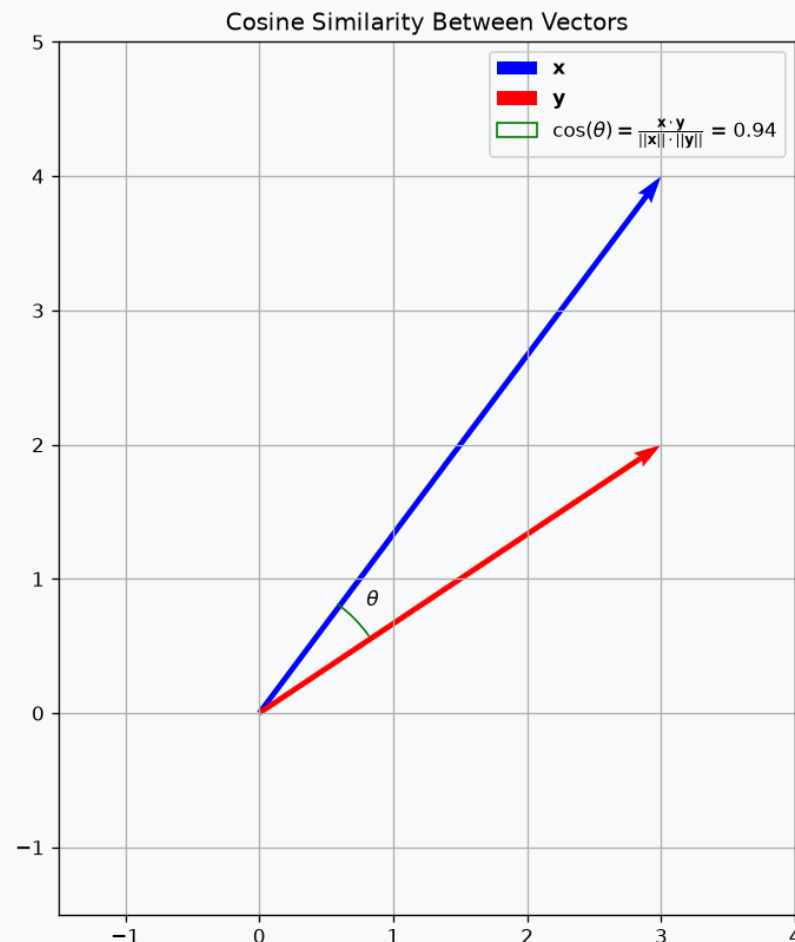
The **L-p normalization** maps a vector to a unit vector, whose L-p norm becomes 1:

$$\hat{\mathbf{x}} = \frac{\mathbf{x}}{\|\mathbf{x}\|_p}$$

A commonly used form is **L2 normalization**, which scales a vector so that its Euclidean length is 1. This projects the vector onto the surface of a unit hypersphere, making it comparable in direction — but no longer in magnitude — to other vectors.



Cosine Similarity: Measuring Orientation



The Euclidean distance between two proportional vectors is in general not zero, $\|\mathbf{x} - \alpha\mathbf{x}\|^2 > 0$ for $\alpha \neq 1$ — and yet $\mathbf{x}$ and $\alpha\mathbf{x}$ are very similar. When we want to compare **the relationships between coordinates rather than their values**, we use the cosine similarity:

$$S_C(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}$$

It computes the cosine of the angle θ between the two vectors, and it is unchanged if one of them is made longer. This matters whenever scale is not informative: if $\mathbf{x}$ and $\mathbf{y}$ are the word counts of two documents of different lengths, we care about the **proportions** of words, while the longer document simply has the larger L2 norm.

The cosine **distance** is defined from it as

$$D_C = 1 - S_C = 1 - \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}$$

+1

Maximum alignment

The vectors point in exactly the same direction

0

Orthogonal

The vectors are perpendicular: no relationship

-1

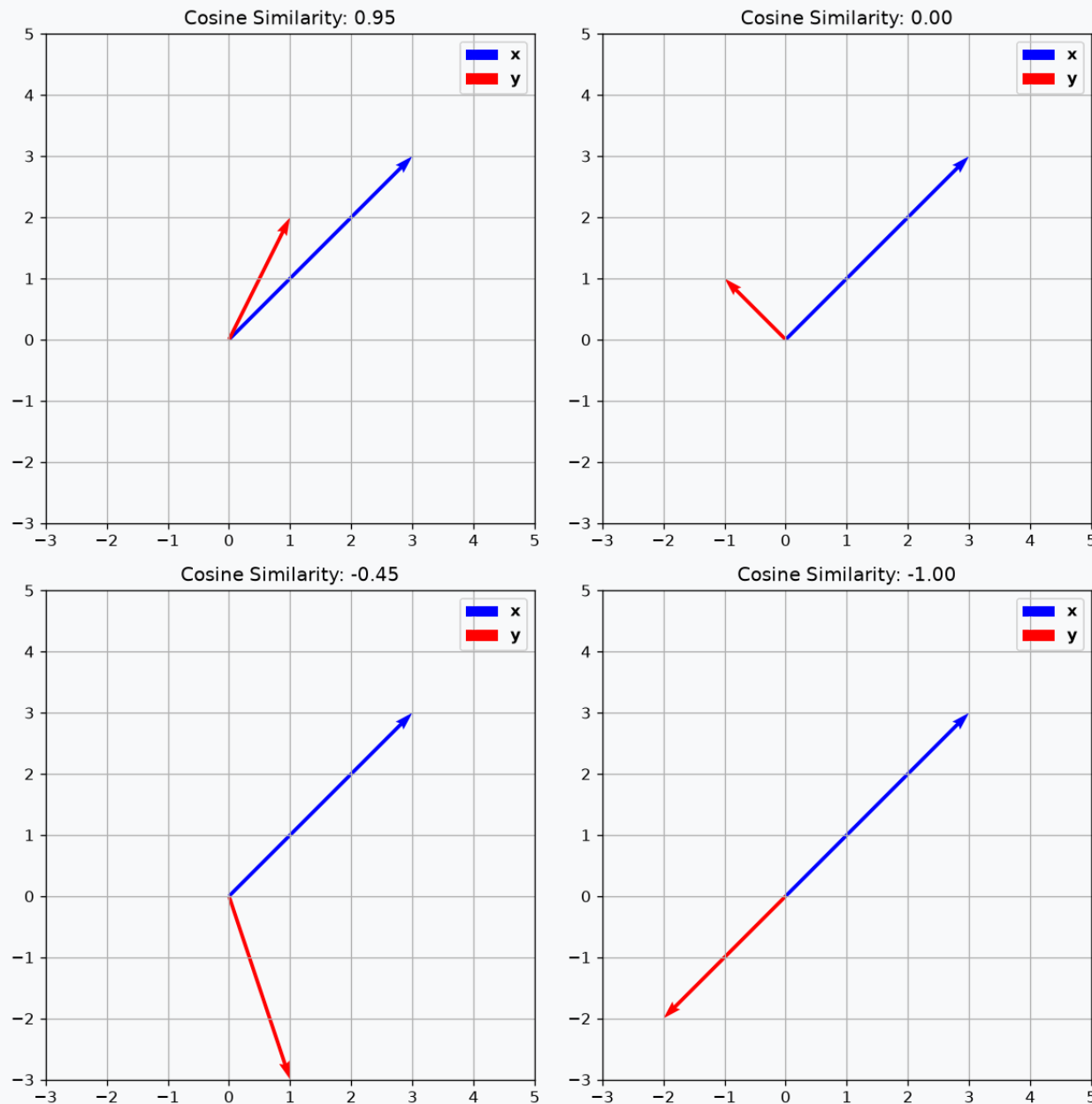
Opposite

The vectors point in exactly opposite directions

- Note that the cosine similarity is **not a metric**: it does not satisfy the triangle inequality.



Examples of Cosine Similarity Values



The same blue vector x , against four different y :

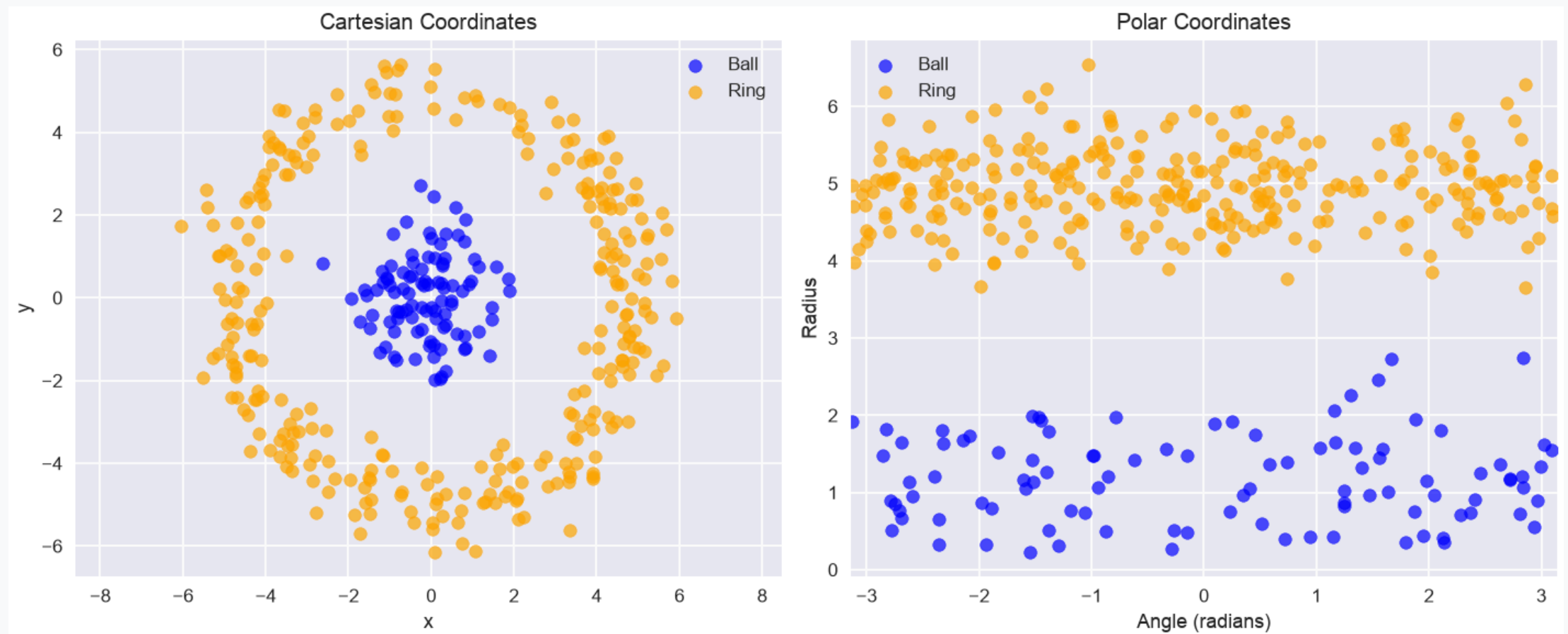
- **0.95** — a small angle: the two vectors are almost aligned, even though one is much shorter than the other.
- **0.00** — a right angle: no relationship between the two directions.
- **-0.45** — an obtuse angle: the vectors disagree more than they agree.
- **-1.00** — exactly opposite directions, whatever the two lengths.

Only the angle enters the measure. Doubling the length of either arrow leaves every one of these four numbers unchanged.



The Power of Coordinate Transformation

The same data can look completely different depending on the coordinate system. The points on the left are not linearly separable; the points on the right — the same points — are:



Taking polar coordinates turns the ball-and-ring problem into one a linear classifier solves. In practice the case-defined `atan2` function is used instead of `arctan`, so that the quadrants are handled correctly.

This demonstrates a fundamental principle: **choosing the right representation can turn a hard problem into a trivial one.**

$$\text{Radius: } r = \sqrt{x^2 + y^2}$$

$$\text{Angle: } \theta = \arctan\left(\frac{y}{x}\right)$$



From Hand-Crafted to Learned Features

Traditional Approach

Hand-craft the feature function using **human intuition and domain knowledge**. It works for cases like polar coordinates, and fails for complex data.

- Requires expert knowledge
- Limited to interpretable features
- Does not scale to complex data

Whether hand-crafted (like r and θ) or learned, the goal of the representation function $f : \mathbb{R}^d \rightarrow \mathbb{R}^m$ is the same: map messy raw data into a feature space where the relationships are simpler and a classifier can find the boundary easily.

- **The key question:** what is the "radius" of a picture of a cat? What is the "angle" of a legal document? For complex real-world data we cannot discover the perfect features by hand — we have to learn them.

Modern Deep Learning

Learn the representation function directly from the data, with neural networks that discover the useful features themselves.

- No manual feature engineering
- Handles complex patterns
- Scales to images, text, audio



Data Processing Inequality

The **Data Processing Inequality (DPI)** states that applying any data transformation, deterministic or stochastic, can never increase the information content about an original random variable. In simple terms: *processing data can only lose or preserve information, never create it.*

If we have a Markov chain $X \rightarrow Y \rightarrow Z$, meaning that Z is conditionally independent of X given Y , then the mutual information satisfies:

$$I(X; Y) \geq I(X; Z)$$

Here $I(X; Y)$ is the shared information between X and Y . As data X is successively transformed into Y and then Z , the information *about the original variable X* can only decrease or stay the same.

This principle has consequences for feature engineering:

Irreversible Loss

If hand-crafted features discard crucial information about the underlying signal, that information is **permanently lost** and cannot be recovered by any later modelling step.

Information Bottleneck

Poor feature design creates an **information bottleneck**, limiting the maximum performance achievable by any model built on those features.

The Advantage of Learned Features

Deep models aim to learn representations that **preserve maximal task-relevant information** directly from raw data, minimising the risk of premature loss.



Black Box Feature Extractors

Modern machine learning uses massive pre-trained models as feature extractors: feed in raw data, get out rich numerical representations that carry semantic meaning.

Text Models

BERT, **GPT** and **Sentence Transformers** convert a raw sentence into a 768-dimensional vector that captures its meaning. The Hugging Face `transformers` library is the standard toolkit.

Image Models

Pre-trained CNNs such as **ResNet**, or **Vision Transformers (ViT)**, turn a raw image into a 2048-dimensional vector representing its visual content, objects and scene.

Audio Models

MFCCs, 1-D CNNs and audio transformers such as **Wav2Vec2** convert a raw waveform into a vector representing acoustic content, speaker identity or spoken words.

In every case the principle is the one above: the model is a representation function f that someone else has already fitted, on far more data than we have.



DLFeat: a Deep Learning Feature Extraction Library



docs gh-pages license MIT

DLFeat: Deep Learning Feature Extraction Library

DLFeat is a Python library designed for easy and modular feature extraction from various data modalities including images, videos, audio, and text. It leverages powerful pre-trained models from libraries like PyTorch, torchvision, Transformers, Sentence-Transformers, and TIMM. The goal is to provide a "black box" tool suitable for educational and research purposes, allowing users to quickly extract meaningful features for data analysis tasks without needing to delve into the complexities of each model's architecture or training.

DLFeat is a Python library for easy, modular feature extraction from images, video, audio and text. It wraps pre-trained models from PyTorch, torchvision, **transformers**, Sentence-Transformers and TIMM behind one interface, so that a course can use them as black boxes without going into each architecture.

Documentation, model zoo and usage guides:

antoninofurnari.github.io/DLFeat



Image models — ResNet, ViT, DINOv2



Video models — Video Swin, R(2+1)D



Audio and multimodal — Wav2Vec2, CLIP

Laboratory: The Image Challenge

We classify **CIFAR-10** images — 32×32 colour pictures in ten classes — with the same simple `KNeighborsClassifier`, in two different feature spaces. 5 000 images for training, 1 000 for testing.



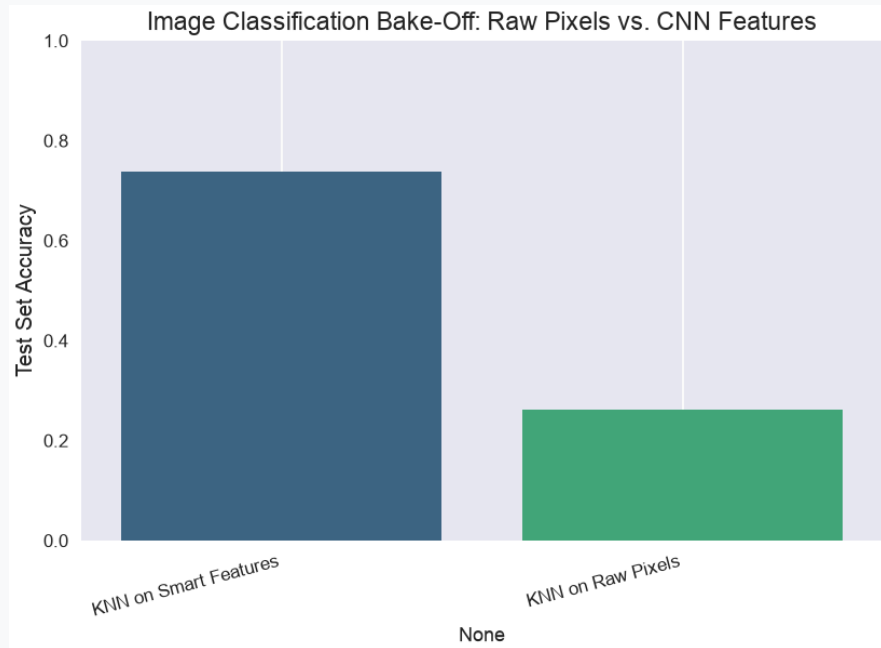
Model 1: the "dumb" baseline

KNN on **raw pixel data**: $32 \times 32 \times 3 = 3\,072$ features, standardised.
A high-dimensional, unstructured space — the setting where we know the curse of dimensionality bites.

Model 2: the "smart" model

A pre-trained **ResNet18**, via `DLFeat`, first turns each image into a **512-dimensional** semantic vector. The same KNN then runs in that space.

Laboratory: The Same Model, Two Feature Spaces



26.20%

KNN on raw pixels

3 072 dimensions, 0.54s

73.90%

KNN on CNN features

512 dimensions, 0.07s to fit

The classifier is *identical*. Only the space changed, and the accuracy nearly tripled.

The pre-trained ResNet has already learned about edges, shapes and objects on ImageNet, so in its 512-dimensional space two pictures of dogs sit close together and far from an aeroplane. In the raw pixel space, "nearest" meant "similar background colour".

The expensive step is the extraction — 32.64s for the 6 000 images — and it is done **once**: the features are cached to disk and reused.

- **The lesson of the lab:** the quality of the representation matters more than the choice of the algorithm. A simple model on smart features beats a sophisticated model on dumb features.

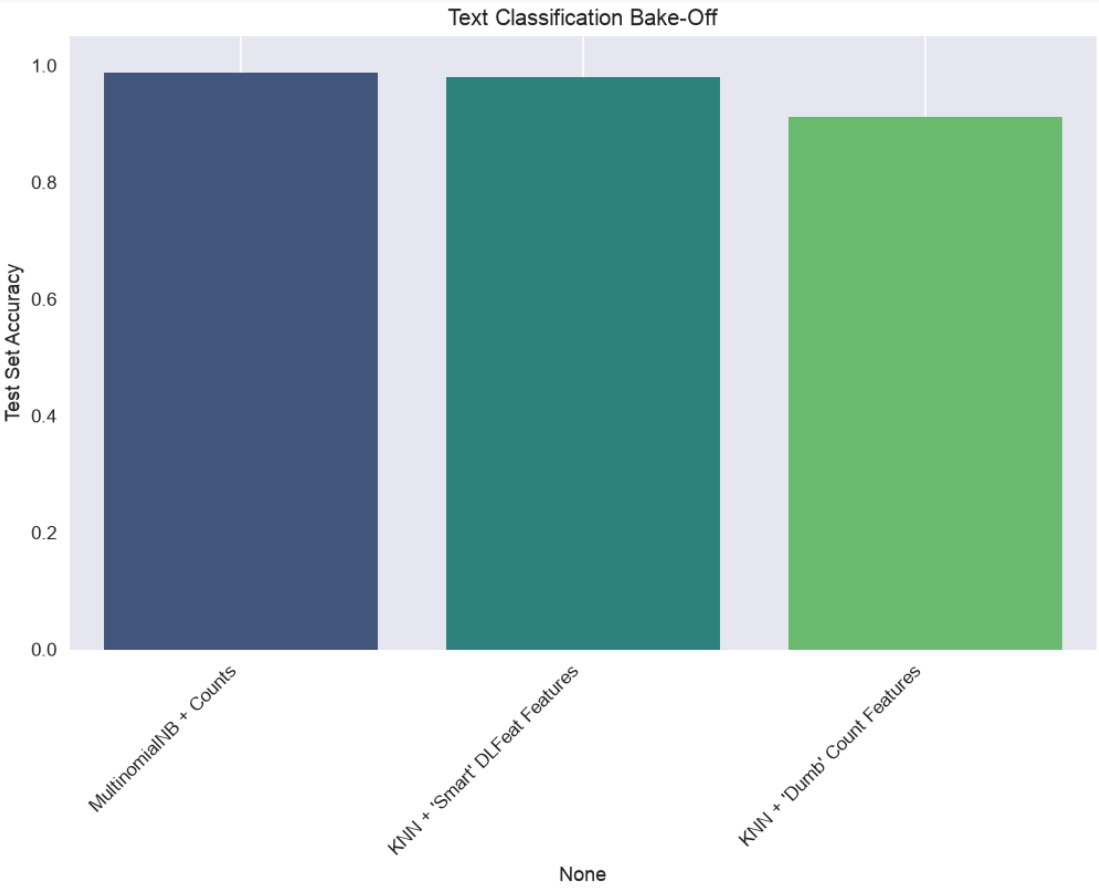


Laboratory: The Same Lesson on Text

Two text bake-offs repeat the experiment with the classifiers of the previous lectures — **multinomial naive Bayes** against **KNN**, and against "smart" KNN: `MultinomialNB` on word counts, `KNeighborsClassifier` on the same counts, and the same KNN on **sentence-transformer** features from `DLFeat`, a 384-dimensional semantic space extracted once in about eight seconds.

Spam detection — 3 900 / 1 672 messages

Model	Test accuracy
MultinomialNB on counts	98.74%
KNN on DLFeat features	97.97%
KNN on counts	91.15%



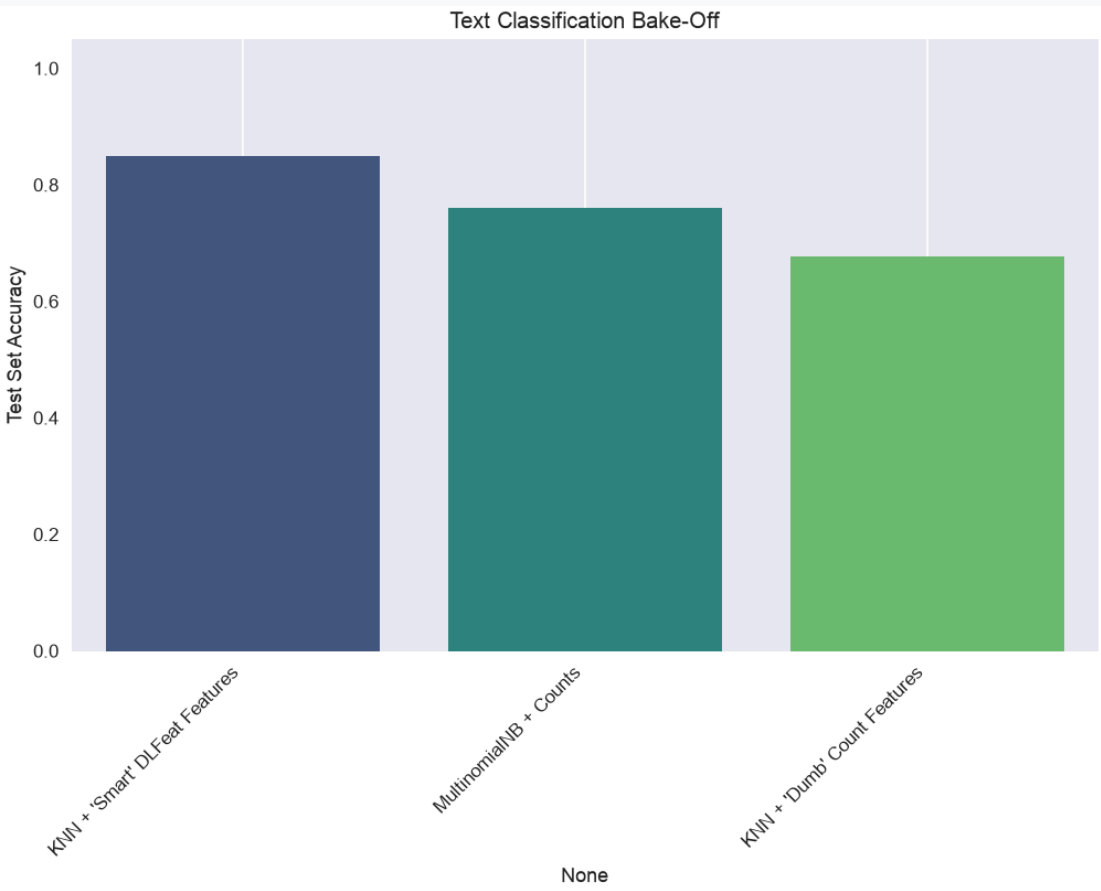
KNN on raw counts comes last in both tasks: thousands of unscaled dimensions are exactly where distances stop meaning anything. The same KNN, on the semantic features, jumps to second place on spam and to **first** on sentiment.

The two tasks are not the same problem. Spam is largely a question of *which words occur*, which is what a multinomial model is built for — the classical model still wins, and fits in a fraction of the time. Sentiment is a question of what the sentence *means*: no count-based model can see that *"I hated this product"* and *"this product is terrible"* agree, since they share no words. There the semantic space is worth nine points.

- The lesson is not "deep features always win". It is that the representation is a modelling choice of the same weight as the algorithm — and that the right one depends on what the task actually turns on.

Sentiment — 700 / 300 product reviews

Model	Test accuracy
KNN on DLFeat features	85.00%
MultinomialNB on counts	76.00%
KNN on counts	67.67%



Conclusions and Next Steps



We Have Explored:

Data as points in a feature space

Norms, and the L-p family

Metrics, and what makes one valid

Cosine similarity and distance

Representation functions, hand-crafted and learned

The data processing inequality

- In the next lectures we look at clustering.

References

- Norms: [en.wikipedia.org/wiki/Norm_\(mathematics\)](https://en.wikipedia.org/wiki/Norm_(mathematics)) · L-p spaces: en.wikipedia.org/wiki/Lp_space
- Metric spaces: en.wikipedia.org/wiki/Metric_space · Cosine similarity: en.wikipedia.org/wiki/Cosine_similarity
- Parts of Chapter 2 of [1]

[1] Goodfellow, Ian, Yoshua Bengio, and Aaron Courville. *Deep learning*. MIT Press, 2016.

- Lecture notes: [Chapter 17 — Data representation](#)

